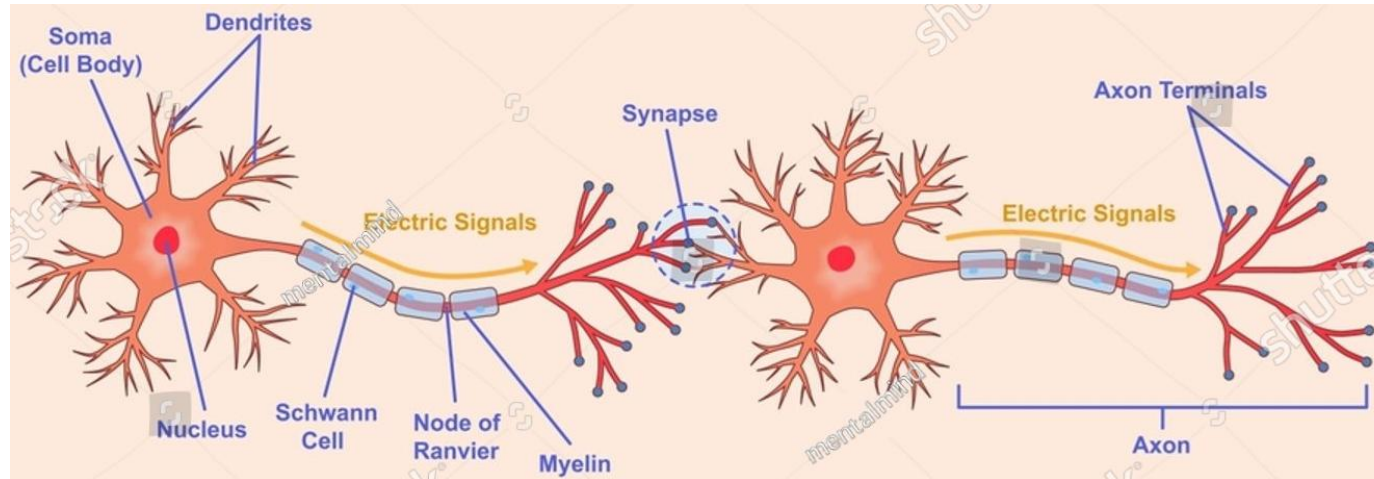


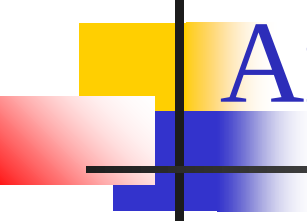
NEURAL NETWORK

Dr. Nguyen Thi Kim Ngan

Biological neuron of the brain



- Soma: the cell body of the neuron, responsible for receiving or generating nerve impulses
- Dendrites: the input nerve fibers that transmit signals to the neuron
- Axon: the output nerve fiber that connects to the dendrites or the soma of another neuron through synapses
- Synapse: the junction that activates or transmits information between neurons



Artificial Neural Network

- Soft computing techniques inspired by the simulation of brain activity (neural network of the brain)

Artificial neuron (Neuron)

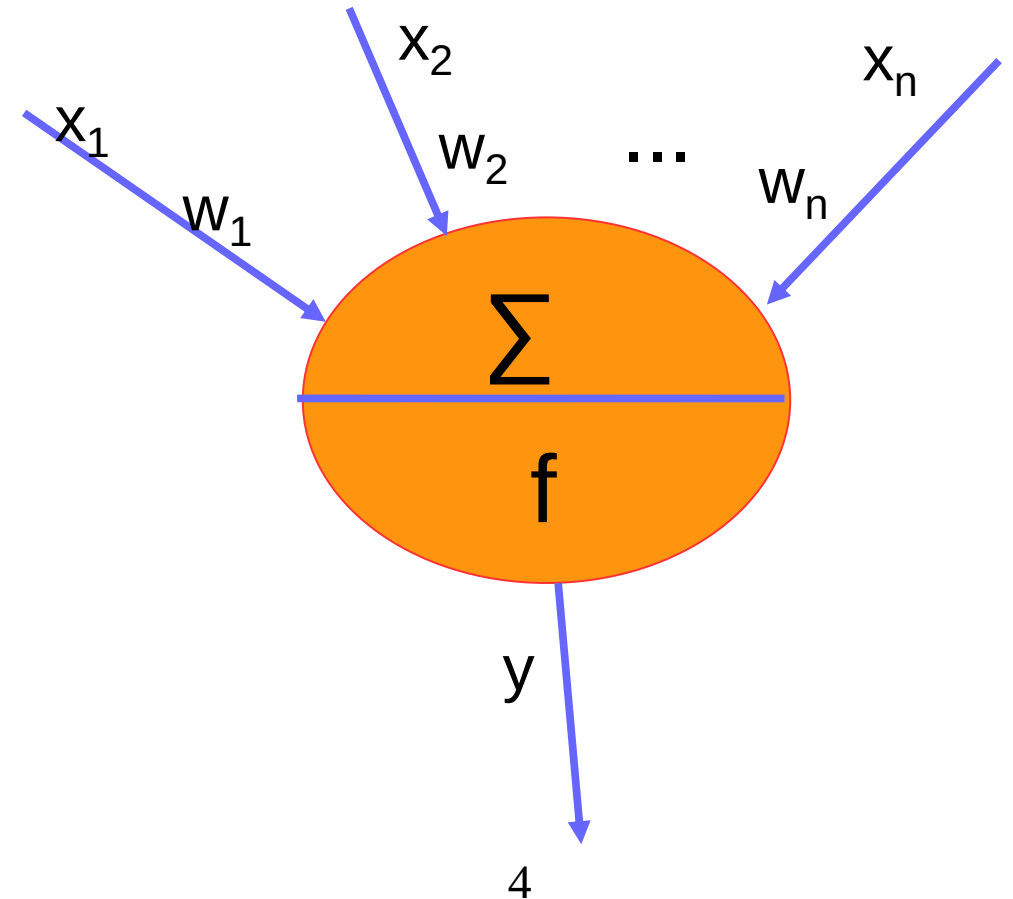
- The total input to a neuron

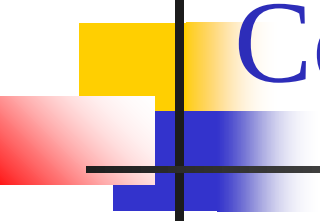
$$\text{Net} = \sum w_i x_i$$

- Output

$$y = f(\text{Net}) = f(\sum w_i x_i)$$

f is called the transfer function or **activation function**





Common activation functions

Sigmoid

$$f(x) = \frac{1}{1 + e^{-x}}$$

Tanh

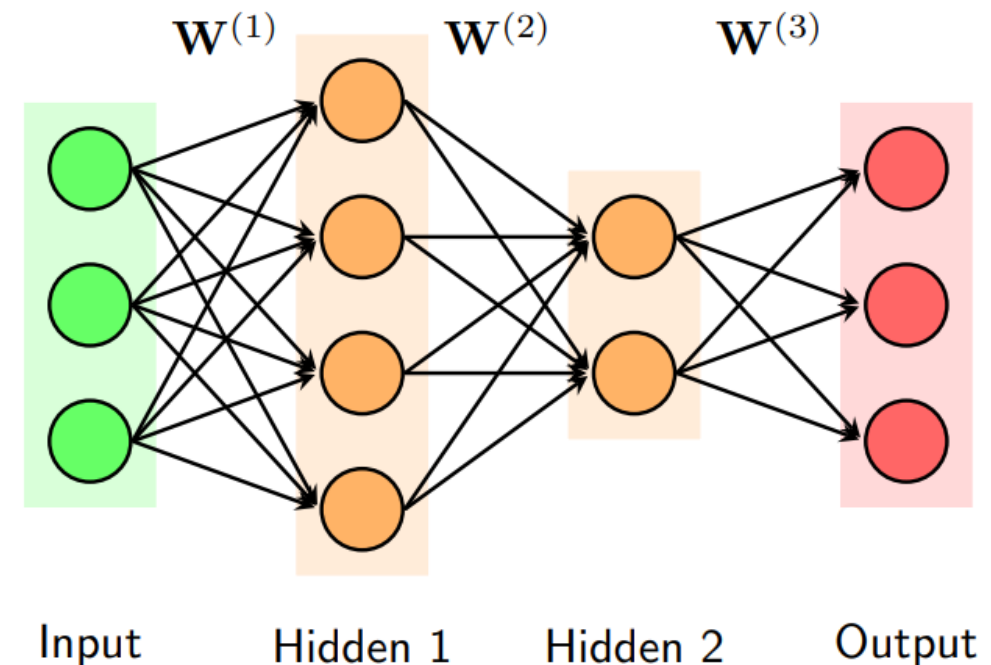
$$f(x) = \tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

ReLu

$$f(x) = \max(0, x)$$

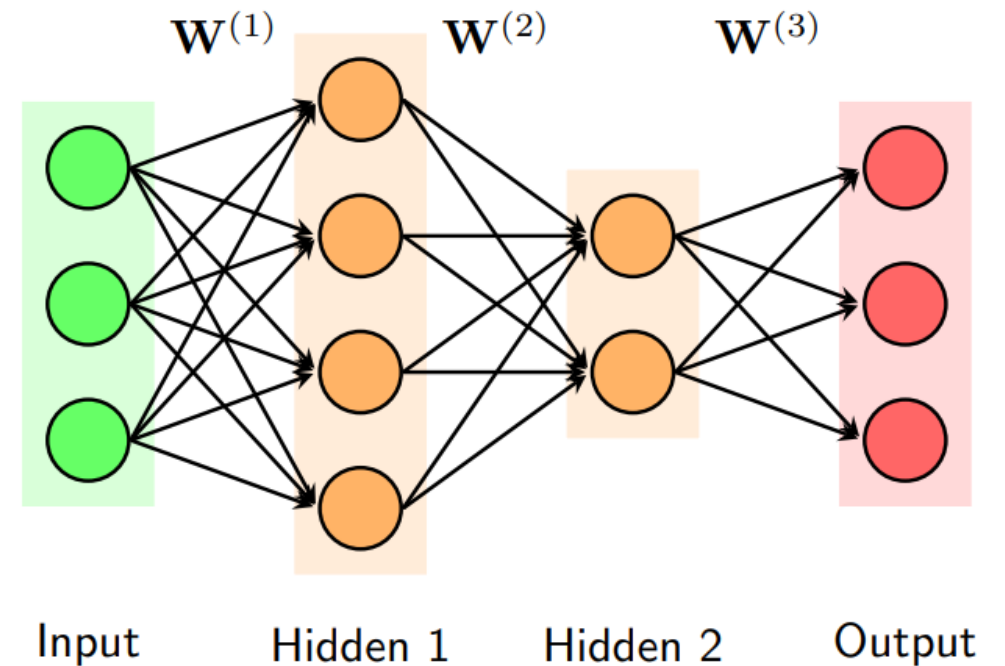
Neural network architecture

- A layer is a collection of neurons arranged vertically
- Three main types of layers:
 - Input layer
 - Hidden layer(s)
 - Output layer
- The total number of layers equals the number of hidden layers plus one
- The number of layers in a Multilayer Perceptron (MLP) is denoted by L . In the figure, $L = 3$



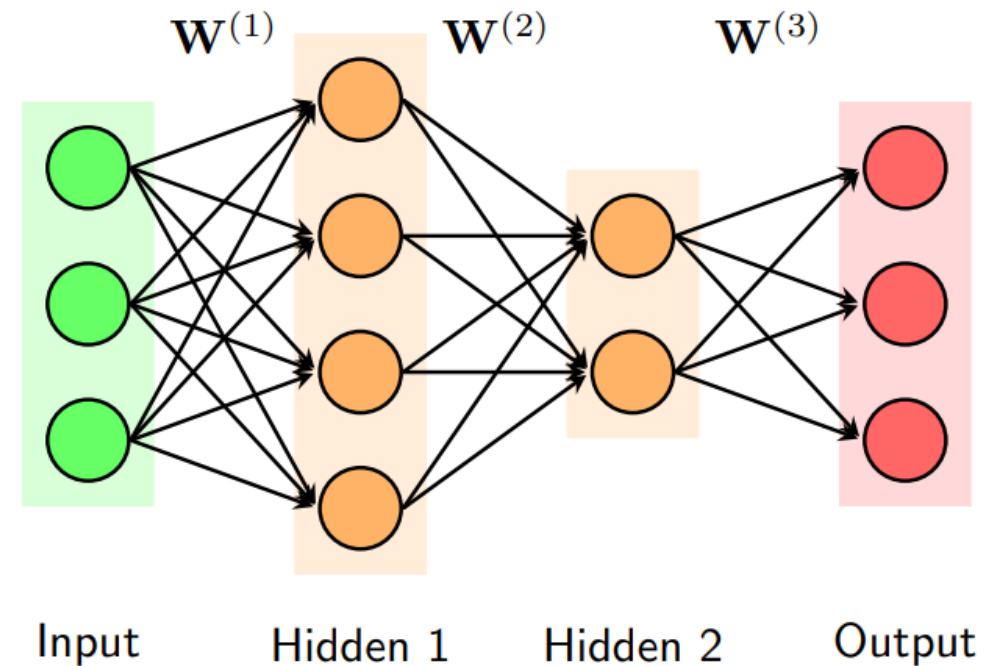
Neural network architecture

- Neuron (Unit/Node): Each circular node is referred to as a neuron
- Each neuron holds a real-valued number

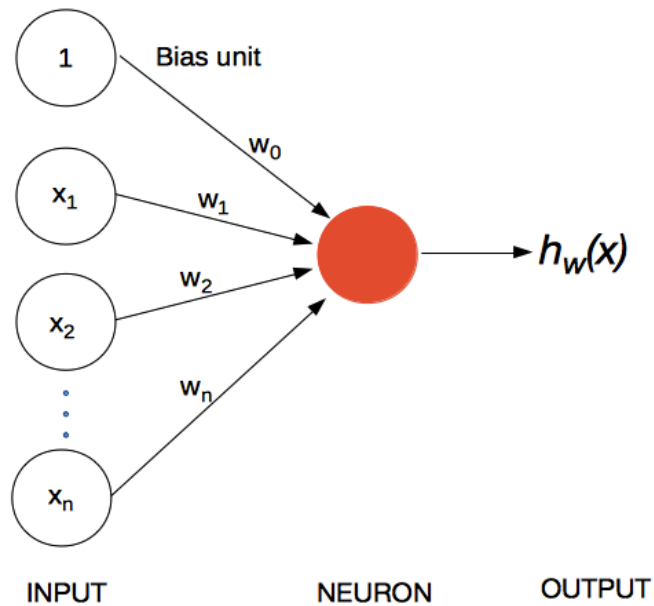


Neural network architecture

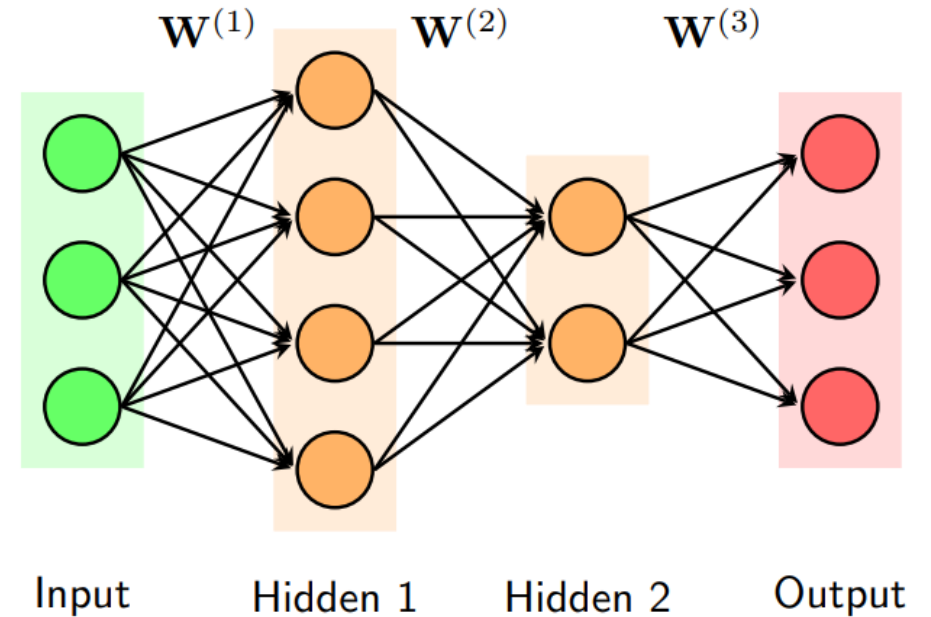
- Weight: A weight is the value assigned to each edge that connects neurons
- Each weight is a real number



Neural network architecture

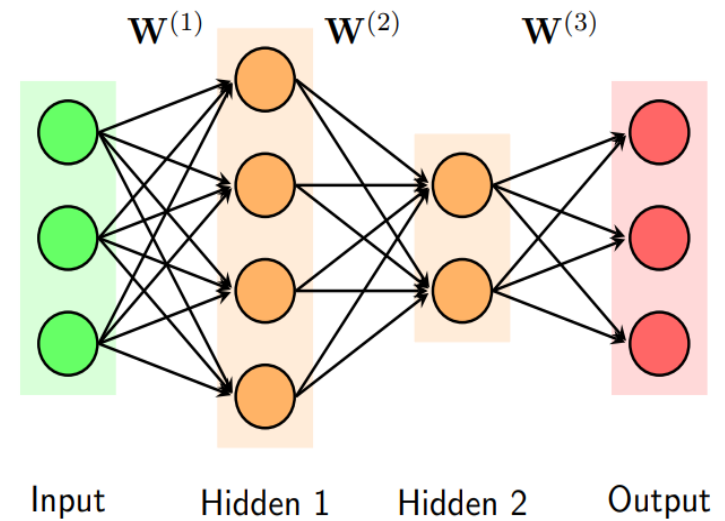
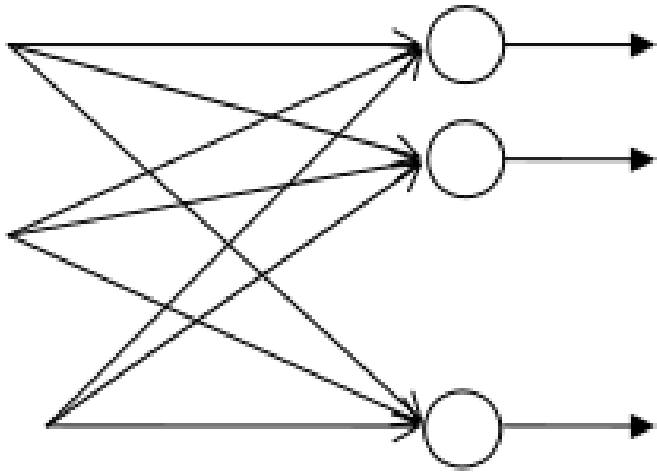


Perceptron
Multilayer neural network



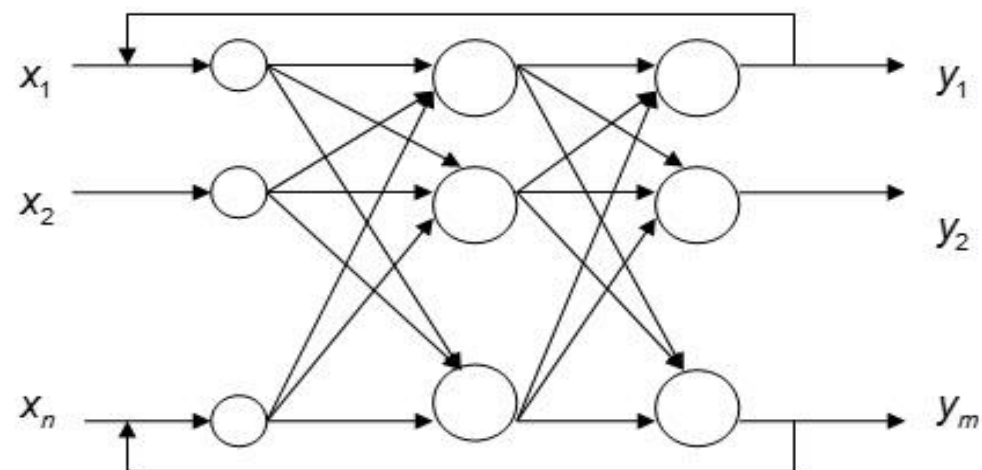
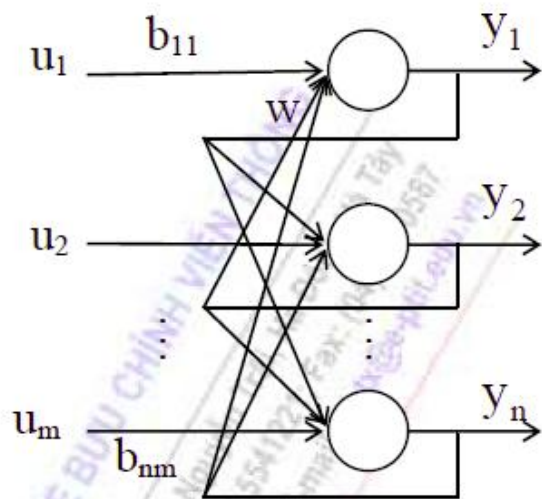
Types of neural networks

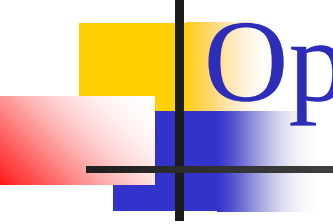
- Feed Forward Neural Networks



Types of neural networks

- Recurrent Neural Networks





Optimization of neural network weights

- **Input:**
 - Training dataset ($X_{\text{train}}, y_{\text{train}}$)
 - Neural network architecture: number of hidden layers, number of neurons per hidden layer
 - Activation function $f(\cdot)$
 - Loss function $J(y, \hat{y})$
- **Output:** Set of all weight matrices $\{W^{(l)}\}$ and bias vectors $\{b^{(l)}\}$ optimized to minimize the loss function
- **Method:**
 - Use Gradient Descent (GD)
 - To apply GD, it is necessary to compute the gradient of the loss function (J) with respect to each weight matrix $W^{(l)}$ and bias vector $b^{(l)}$

Notations

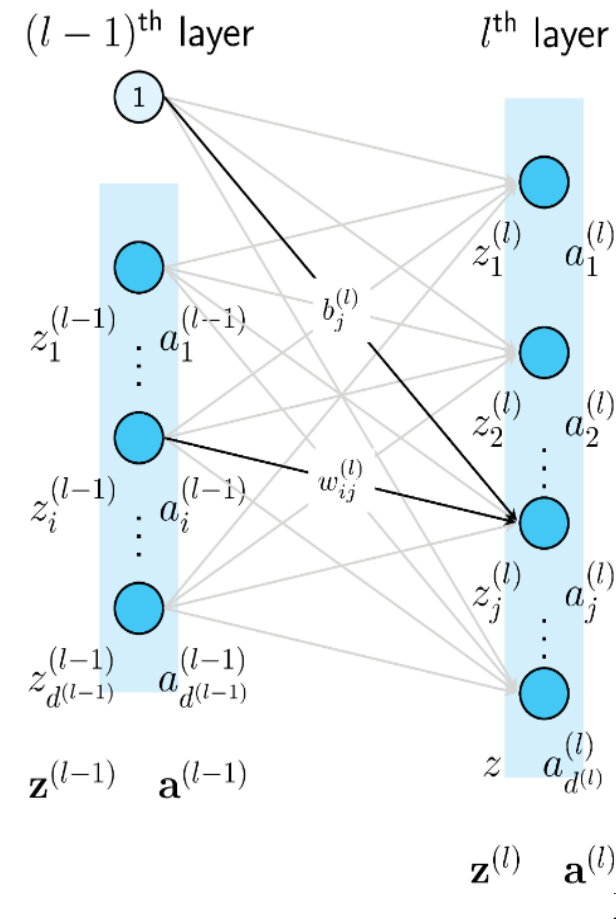
Each output of a unit (except for the input units) is calculated based on the formula:

$$a_i^{(l)} = f(\mathbf{w}_i^{(l)T} \mathbf{a}^{(l-1)} + b_i^{(l)})$$

where $f(\cdot)$ is an activation function

In vector form, the above expression can be written as:

$$\mathbf{a}^{(l)} = f(\mathbf{W}^{(l)T} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)})$$



$$\mathbf{W}^{(l)} \in \mathbb{R}^{d^{(l-1)} \times d^{(l)}}$$

$$\mathbf{b}^{(l)} \in \mathbb{R}^{d^{(l)} \times 1}$$

$$z_j^{(l)} = \mathbf{w}_j^{(l)T} \mathbf{a}^{(l-1)} + b_j^{(l)}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)T} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$$

Feedforward

Feedforward: Compute from the beginning to the end of the network to calculate the predicted output $\hat{y}$ for a given input $\mathbf{x}$

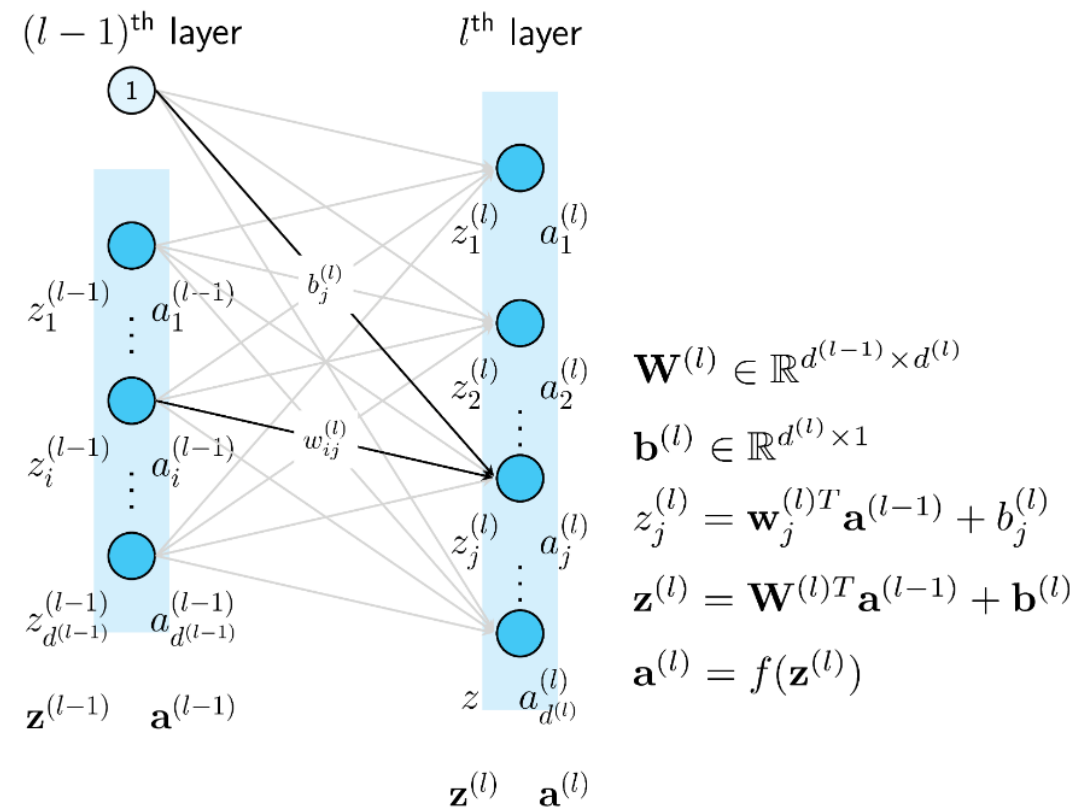
$$\mathbf{a}^{(0)} = \mathbf{x}$$

$$z_i^{(l)} = \mathbf{w}_i^{(l)T} \mathbf{a}^{(l-1)} + b_i^{(l)}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)T} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad l = 1, 2, \dots, L$$

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}), \quad l = 1, 2, \dots, L$$

$$\hat{y} = \mathbf{a}^{(L)}$$





Backpropagation

Let $J(W, b, X, Y)$ is the loss function

- W, b : set of all weight matrices between layers and biases
- X, Y : are the training data pairs, with each column corresponding to a sample.
- To apply GD, it is necessary to compute:

$$\frac{\partial J}{\partial \mathbf{W}^{(l)}}; \frac{\partial J}{\partial \mathbf{b}^{(l)}}, \quad l = 1, 2, \dots, L$$

Backpropagation

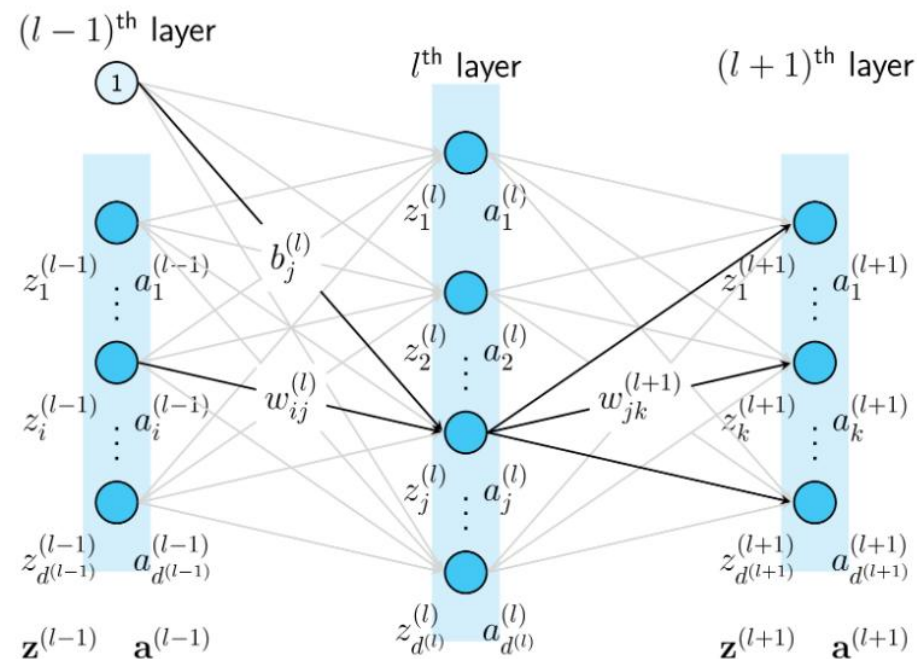
$$\frac{\partial J}{\partial w_{ij}^{(L)}} = \frac{\partial J}{\partial z_j^{(L)}} \cdot \frac{\partial z_j^{(L)}}{\partial w_{ij}^{(L)}} = e_j^{(L)} a_i^{(L-1)}$$

$$e_j^{(L)} = \frac{\partial J}{\partial z_j^{(L)}}$$

$$\frac{\partial z_j^{(L)}}{\partial w_{ij}^{(L)}} = a_i^{(L-1)}$$

$$z_j^{(L)} = \mathbf{w}_j^{(L)T} \mathbf{a}^{(L-1)} + b_j^{(L)}$$

$$\frac{\partial J}{\partial b_j^{(L)}} = \frac{\partial J}{\partial z_j^{(L)}} \cdot \frac{\partial z_j^{(L)}}{\partial b_j^{(L)}} = e_j^{(L)}$$



$$\mathbf{W}^{(l)} \in \mathbb{R}^{d^{(l-1)} \times d^{(l)}}$$

$$\mathbf{b}^{(l)} \in \mathbb{R}^{d^{(l)} \times 1}$$

$$z_j^{(l)} = \mathbf{w}_j^{(l)T} \mathbf{a}^{(l-1)} + b_j^{(l)}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)T} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$$

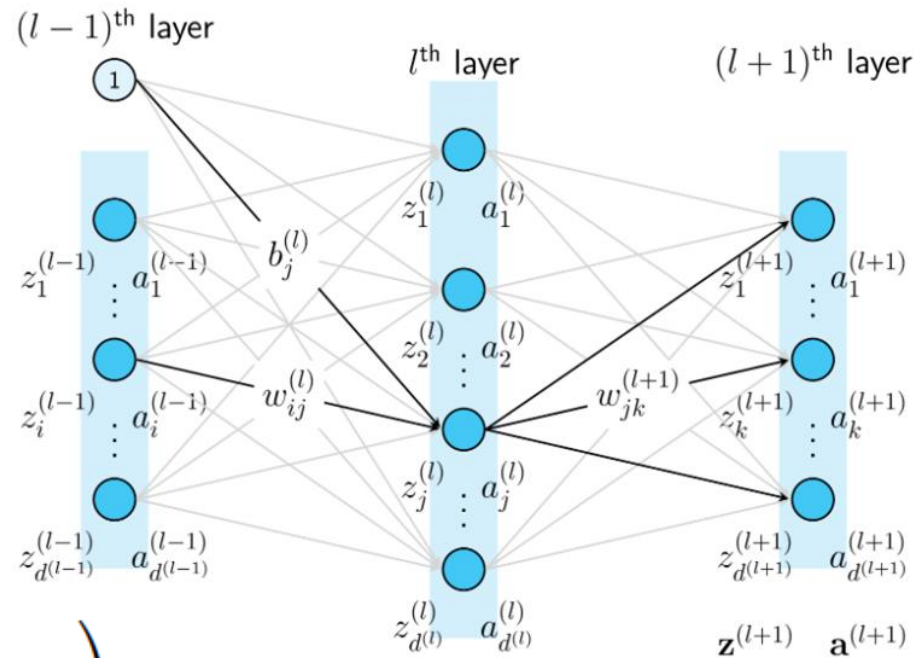
Backpropagation

$$\begin{aligned}\frac{\partial J}{\partial w_{ij}^{(l)}} &= \frac{\partial J}{\partial z_j^{(l)}} \cdot \frac{\partial z_j^{(l)}}{\partial w_{ij}^{(l)}} \\ &= e_j^{(l)} a_i^{(l-1)}\end{aligned}$$

$$e_j^{(l)} = \frac{\partial J}{\partial z_j^{(l)}} = \frac{\partial J}{\partial a_j^{(l)}} \cdot \frac{\partial a_j^{(l)}}{\partial z_j^{(l)}}$$

$$= \left(\sum_{k=1}^{d^{(l+1)}} \frac{\partial J}{\partial z_k^{(l+1)}} \cdot \frac{\partial z_k^{(l+1)}}{\partial a_j^{(l)}} \right) f^{(l)'}(z_j^{(l)}) = \left(\sum_{k=1}^{d^{(l+1)}} e_k^{(l+1)} w_{jk}^{(l+1)} \right) f^{(l)'}(z_j^{(l)})$$

$$\text{vì } a_j^{(l)} = f^{(l)}(z_j^{(l)}).$$



$$\mathbf{W}^{(l)} \in \mathbb{R}^{d^{(l-1)} \times d^{(l)}}$$

$$\mathbf{b}^{(l)} \in \mathbb{R}^{d^{(l)} \times 1}$$

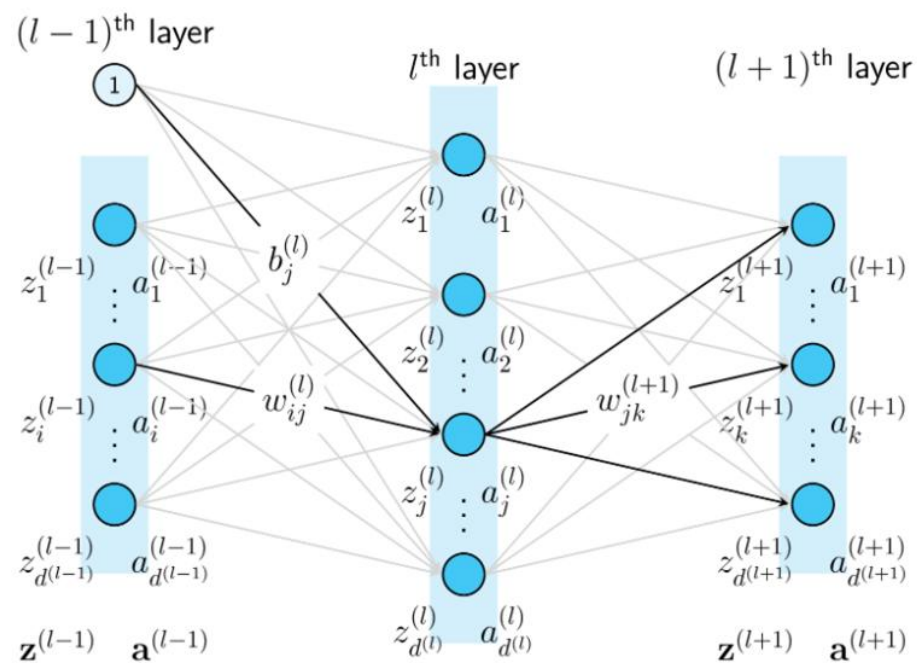
$$z_j^{(l)} = \mathbf{w}_j^{(l)T} \mathbf{a}^{(l-1)} + b_j^{(l)}$$

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)T} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$$

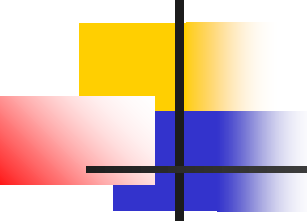
$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)})$$

Backpropagation

$$\frac{\partial J}{\partial b_j^{(l)}} = e_j^{(l)}$$



$$\begin{aligned} \mathbf{W}^{(l)} &\in \mathbb{R}^{d^{(l-1)} \times d^{(l)}} \\ \mathbf{b}^{(l)} &\in \mathbb{R}^{d^{(l)} \times 1} \\ z_j^{(l)} &= \mathbf{w}_j^{(l)T} \mathbf{a}^{(l-1)} + b_j^{(l)} \\ \mathbf{z}^{(l)} &= \mathbf{W}^{(l)T} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)} \\ \mathbf{a}^{(l)} &= f(\mathbf{z}^{(l)}) \end{aligned}$$



GD method

1. Initialize all weights $W^{(l)}$ and biases $b^{(l)}$ with small random values.
2. Repeat until convergence (or max epochs):
 - a. Forward propagation $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$
 - b. Compute loss $\mathcal{L} = \text{Loss}(y, \hat{y})$
 - c. Backward propagation:
 - Compute error term at output layer: $\delta^{(L)} = \frac{\partial \mathcal{L}}{\partial a^{(L)}} \cdot f'(z^{(L)})$
 - For hidden layers $l = L-1, \dots, 1$ $\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \cdot f'(z^{(l)})$
 - d. Computer gradients: $\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)}(a^{(l-1)})^T, \quad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)}$
 - e. Update parameters (Gradient Descent): $W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}}, \quad b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial b^{(l)}}$